

Reconocimiento de Marcha para Asistencia Automatizada

Modelos Utilizados · Arquitecturas · Scripts · Modelos Descartados

Proyecto: ProyectoChino | Fecha: 2026-05-06 | GPU: GTX 1650 4 GB

1. MODELOS EN PRODUCCIÓN

1.1 YOLOv11n — Detección de Personas

YOLOv11n es el detector de objetos que localiza a la persona en cada frame de la cámara antes de cualquier otro procesamiento.

Parámetro	Valor
Peso del modelo	6 MB
Confianza mínima (conf)	0.35
IoU threshold	0.50
Latencia típica	~10–20 ms / frame
Selección de bbox	Mayor área (persona más cercana)

¿Cómo se usó? Corre en cada frame a 15 fps. Cuando detecta múltiples personas, el pipeline selecciona la de mayor bounding box (más cercana a la cámara). Su salida alimenta directamente al FSM Tracker.

¿Por qué YOLOv11n y no otro? Cabe en 4 GB de VRAM junto con los demás modelos, tiene latencia menor a 20 ms, y la precisión de detección de personas en entornos controlados es más que suficiente. No se requería un modelo más pesado.

1.2 RVM MobileNetV3 — Segmentación de Siluetas

Robust Video Matting (RVM) con backbone MobileNetV3 convierte cada frame en una silueta binaria de 64×44 px — la entrada que GaitBase necesita.

Parámetro	Valor
Backbone	MobileNetV3
Downsample ratio	0.25 (480×270 interno)
Resolución de salida silueta	64 × 44 px, binaria
Latencia típica	~30–40 ms / frame
Estado interno	Recurrente — se resetea con el FSM

¿Cómo se usó? Solo se activa cuando el tracker está en estado ACTIVE. El *downsample_ratio=0.25* permite procesar a resolución reducida internamente para ahorrar VRAM sin pérdida de calidad relevante en la silueta resultante.

¿Por qué RVM y no BackgroundSubtractor? Un sustractor de fondo clásico falla ante iluminación cambiante, sombras o fondos no perfectamente estáticos. RVM es robusto a estas variaciones y produce siluetas limpias incluso con condiciones imperfectas.

¿Cómo ayudó al proyecto? Es el paso de conversión video→silueta. Sin siluetas de buena calidad, GaitBase no puede extraer patrones de marcha confiables. La calidad del segmentador limita directamente las métricas finales del sistema.

1.3 RTMPose — Extracción de Keypoints

RTMPose detecta 17 keypoints del esqueleto (hombros, codos, muñecas, caderas, rodillas, tobillos) sobre cada frame. Aunque la rama de pose fue descartada del score de identidad, RTMPose cumple dos roles en el proyecto.

Rol	Descripción
Fase 1 (offline)	Generar archivos .npy de keypoints para todo el dataset
Pipeline en vivo	Quality gating: descartar siluetas donde la pose es incoherente o el sujeto está parcialmente fuera de cuadro

1.4 GaitBase (ResNet9 Fine-tuned) — Modelo Central de Identidad

GaitBase es el núcleo del sistema. Convierte una secuencia de siluetas en un embedding de identidad que permite comparar personas.

Origen: Baseline del framework OpenGait (Fan et al., CVPR 2023 Highlight). Preentrenado en Gait3D — 4,000 sujetos en condiciones no controladas.

Arquitectura interna:

```
Entrada: secuencia de siluetas [T=60 frames × 64 × 44 px, binarias]
↓ SetBlockWrapper
Aplica ResNet9 frame a frame (pesos compartidos)
ResNet9: canales [64→128→256→512], capas [1,1,1,1], strides [1,2,2,1]
↓ Temporal Pooling
PackSequenceWrapper(torch.max) → máximo temporal → (512, H', W')
↓ HorizontalPoolingPyramid (HPP)
bin_num=[16] → divide en 16 bandas horizontales (cabeza→pies)
↓ SeparateFCs
512 → 256 dims por cada una de las 16 bandas
↓ BNNeck (BatchNorm Bottleneck)
class_num=13 (sujetos de entrenamiento)
Salida: embedding 256 × 16 = 4,096 dims, L2-normalizado
→ similitud coseno contra galería
```

Componente	Función
ResNet9	Extrae características espaciales de cada silueta individual
SetBlockWrapper	Aplica ResNet9 sobre cada frame de la secuencia con pesos compartidos
Temporal Pooling	Condensa toda la secuencia en un solo descriptor (máximo temporal)
HPP (16 bandas)	Captura detalles locales: dinámica de cabeza, torso, piernas y pies por separado
BNNeck	Normaliza antes de la clasificación, estabiliza el espacio de embeddings
L2-normalizado	Permite usar similitud coseno como métrica de distancia (rango [-1, 1])

¿Por qué ResNet9 y no ResNet50? Las siluetas son imágenes binarias de 64×44 px — mucho más simples que imágenes RGB naturales. ResNet9 con 4 capas es suficiente para extraer representaciones discriminativas y cabe en 1.25 GB de VRAM durante entrenamiento.

¿Por qué HPP? Dividir la silueta en 16 bandas horizontales permite que el modelo aprenda la dinámica de cada región del cuerpo de forma independiente. El movimiento de piernas no interfiere con el de brazos en el embedding.

Checkpoint en producción:

checkpoints/finetune/gaitbase_ft_multisession_best_iter1200.pt
Tamaño: ~27 MB | Iteración de parada anticipada: 1,200 / 1,500 máx.
Entrenado con: 13 sujetos × s1 + s2 combinadas (~51 secuencias)

Métrica	Valor
Rank-1 NN (normal → normal)	94.4% (17/18 correctos)
Rank-1 NR (normal → rápido)	94.7% (18/19 correctos)
Rank-5	100%
EER open-set	8.11%
TAR @ FAR=0%	86.49% (32/37 genuinos aceptados)
Umbral τ	0.9847
VRAM pico entrenamiento	1.25 GB
Tiempo de entrenamiento	~177 min en GTX 1650

2. MODELOS DESCARTADOS

2.1 GaitBase Pretrained sin Fine-tune

La primera evaluación fue correr el checkpoint preentrenado en Gait3D directamente sobre nuestro dataset, sin ninguna adaptación.

Métrica	Resultado
EER	48.65% (prácticamente aleatorio)
TAR @ FAR=0%	16%

¿Por qué falló? Gait3D tiene 4,000 sujetos en exteriores con múltiples cámaras y ángulos variables. El dominio de nuestro dataset — 19 personas específicas, 90° lateral fijo, indoor — es completamente distinto. El modelo no puede generalizar sin adaptación.

2.2 GaitBase Fine-tune solo con Sesión 1 (s1)

Se hizo fine-tuning usando únicamente los datos de la sesión 1 (paso normal) para los 13 sujetos de entrenamiento.

- Resultado en closed-set: Rank-1 aceptable dentro de s1.
- Problema crítico: degradación severa en cross-session (s1→s2).

¿Por qué falló? El modelo aprendió la diferencia entre personas basándose parcialmente en la ropa de cada uno en s1. Cuando se evaluó con s2 (ropa diferente, velocidad diferente), el modelo no generalizó. Este resultado motivó el entrenamiento multi-sesión.

2.3 SkeletonGait++ (SGPP) — Fusión Silueta + Pose

SkeletonGait++ es una arquitectura multimodal publicada en AAAI 2024. Combina una rama de silueta con una rama de skeleton map (heatmap de keypoints) con pesos de fusión aprendibles.

¿Cómo se intentó? Se inicializó con los pesos del GaitBase fine-tuned (multisesión) y se añadió la rama de pose. Se entrenó optimizando el peso α de fusión entre 0.0 (solo pose) y 1.0 (solo silueta).

Experimento	α óptimo	EER	TAR@FAR=0%
SGPP fusión tardía	1.0	8.11%	86.49%
ST-GCN Lite fusión	1.0	8.11%	86.49%

Resultado: $\alpha^*=1.0$ en todos los experimentos. El optimizador asignó peso cero a la rama de pose en todos los casos. Silueta sola gana.

Cuatro razones técnicas del fracaso:

- **Ángulo lateral 90°:** Las extremidades contralaterales se solapan en la imagen. Los keypoints de RTMPose ven poca diferencia entre sujetos a este ángulo. La silueta ya captura prácticamente toda la información de marcha visible.
- **Dataset insuficiente para pose:** Solo ~51 secuencias de entrenamiento (13 sujetos $\times$ ~4 tipos). Las redes de pose necesitan miles de ejemplos para no sobreajustarse al ruido de detección.
- **Solo 2 sesiones:** La red de pose aprende la diferencia s1 vs s2 (velocidad, ropa) en lugar de la dinámica real de marcha. El modelo aprende ruido del detector de poses como 'firma de identidad'.
- **Evidencia cuantitativa directa:** El optimizador nos dijo explícitamente que la pose no aporta. Corregir a ricardomora con SGPP rompió otros pares — el trade-off no fue ventajoso.

Estado final de la pose en el pipeline: RTMPose sigue corriendo en runtime para quality gating (descartar siluetas mal extraídas), pero su salida NO entra en el score de similitud de identidad.

2.4 ST-GCN Lite — Solo Pose

Versión ligera de Spatial-Temporal Graph Convolutional Network. Modela relaciones entre articulaciones del esqueleto a lo largo del tiempo usando grafos.

- Entrenado únicamente con keypoints RTMPose (sin silueta).
- Resultado: $\alpha^*=1.0$ en fusión. Misma conclusión que SGPP.

¿Por qué falló? Además de las razones ya mencionadas, un GCN espaciotemporal tiene muchos parámetros relativos a 51 secuencias de entrenamiento. El sobreajuste fue inmediato.

3. HIPERPARÁMETROS DE ENTRENAMIENTO

Parámetro	Valor	Justificación
Optimizador	SGD	Estándar en OpenGait, más estable con TripletLoss
Learning Rate	0.01	Configuración original de GaitBase
Momentum	0.9	Estándar en SGD
Weight Decay	5×10^{-4}	Regularización — necesaria con dataset pequeño
Batch (P×K)	P=4, K=2	4 identidades $\times$ 2 secuencias = 8 muestras / iter
Frames por clip	30	Ventana aleatoria — actúa como data augmentation
Iteraciones máx.	1,500	Early stop activado a iter 1,200
LR Milestones	[750, 1250]	Decay $\times 0.1$ en cada milestone
Triplet Margin	0.2	Margen para TripletLoss
CE Scale / Smooth	16 / 0.1	Label smoothing para dataset pequeño
AMP float16	Activado	Necesario para caber en GTX 1650 4 GB

Función de pérdida combinada:

```
Loss = TripletLoss(margin=0.2) + CrossEntropyLoss(scale=16, smoothing=0.1)
```

TripletLoss → acerca embeddings de la misma persona, aleja los de distintas

CrossEntropy → el modelo aprende a clasificar entre las 13 clases de train

Label smooth → evita confianza extrema, mejora generalización en dataset pequeño

4. SCRIPTS DEL PROYECTO

4.1 Preprocesamiento — Fases 0 a 2

Script	Qué hace
audit_raw_dataset.py	Cuenta secuencias, verifica calidad y reporta estadísticas del dataset crudo. Primera ejecución del proyecto.
extract_all.py	Corre RVM (siluetas PNG) + RTMPose (keypoints NPY) sobre todos los videos crudos.
pack_to_pkl.py	Empaqueta los PNGs/NPYs en archivos .pkl formato OpenGait (una secuencia por archivo).
merge_sessions_pkl.py	Fusiona PKLs de s1 y s2 en data/pkl_multisession/ para entrenamiento cross-session.
build_partition_json.py	Genera partition_finetune.json con los índices train/val/test.
define_splits.py	Genera configs/splits.yaml con los 13/3/3 sujetos por split (seed=42, immutable).
validate_opengait_dataloader.py	Verifica que el dataloader lee los PKLs correctamente antes de entrenar.

4.2 Entrenamiento — Fases 3 y 4

Script	Estado	Qué hace
finetune_gaitbase.py	Archivado	Fine-tune de GaitBase solo con s1. Resultado descartado por degradación con s2.
finetune_gaitbase_multisession.py	✓ Producción	Fine-tune con s1+s2 combinadas. Produce gaitbase_ft_multisession_best_iter.pt
finetune_skeletongaitpp.py	Descartado	Fine-tune de SkeletonGait++. $\alpha^*=1.0$, silueta sola gana.
train_stgcn.py	Descartado	Entrenamiento de ST-GCN Lite solo con keypoints. Sobreajuste severo.
smoke_test_gaitbase.py	Utilidad	Test rápido: carga modelo, pasa un batch, verifica que no hay errores de dimensión.

4.3 Evaluación — Fases 3 a 5

Script	Estado	Qué hace
cross_session_eval.py	✓ Activo	Evaluación closed-set: rank-1, rank-5, matriz de confusión en los 19 sujetos.
openset_eval.py	✓ Activo	Calibración de τ : genera curva DET, calcula EER y TAR@FAR=0%. Produce gaitbase_ft_multisession_tau.pt
openset_eval_skeletongaitpp.py	Descartado	Evaluación open-set de SGPP. Confirma $\alpha^*=1.0$.
openset_eval_fusion.py	Descartado	Evaluación de fusión con peso α variable. Resultado: $\alpha^*=1.0$ en todos los casos.
openset_eval_fusion_pose.py	Descartado	Variante de fusión tardía silueta+pose. Mismos resultados.

4.4 Pipeline en Vivo — Fase 6

Script / Módulo	Qué hace
build_gallery.py	Construye la galería: pasa s1 y s2 de los 19 sujetos por GaitBase. Guarda embeddings.npy ($19 \times K \times 256 \times 1$)
infer_video.py	Inferencia offline sobre un .mp4. Genera CSV de identidades con timestamps. Útil para testing sin cámara en mano.
infer_live.py ← PRINCIPAL	Script de producción. Abre C920, corre el pipeline completo en tiempo real, muestra UI OpenCV y escribe log.

4.5 Módulos Internos — src/pipeline/

Módulo	Responsabilidad
capture.py	Lee frames de cámara o archivo de video con OpenCV.
detect.py	Wrapper de YOLOv11n. Devuelve bounding box de mayor área.
track.py	FSM de 4 estados: IDLE → WARMING → ACTIVE → RESET. Warmup 15 frames, reset tras 15 frames sin detección.
segment.py	Wrapper de RVM MobileNetV3. Devuelve silueta binaria 64×44.
seq_buffer.py	FIFO con ventana=60 frames y stride=30. Emite una secuencia completa cada 30 frames (2 s a 15 fps).
embed.py	Carga GaitBase fine-tuned. Convierte secuencia de siluetas en embedding 256×16, L2-normalizado.
match.py	Compara embedding contra galería usando similitud coseno. Devuelve mejor match y similitud. Aplica umbral τ .
confirm.py	Exige N=2 identificaciones consecutivas iguales por encima de τ antes de confirmar la identidad.
log.py	Escribe fila en CSV con timestamp, subject, mean_sim, n_sequences, frames_used y session_id.

5. RESUMEN COMPARATIVO DE MODELOS

Modelo	Tipo	EER	TAR@FAR=0%	Estado
GaitBase pretrained (sin fine-tune)	Silueta	48.65%	16%	■ Descartado
GaitBase FT s1 solo	Silueta	—	Degrada CS	■ Descartado
GaitBase FT s1+s2	Silueta	8.11%	86.49%	✓ Producción
SkeletonGait++	Sil.+Pose	8.11%	86.49%	■ $\alpha^*=1.0$
ST-GCN Lite	Solo Pose	8.11%	86.49%	■ $\alpha^*=1.0$

Los modelos multimodales (SGPP, ST-GCN) producen exactamente las mismas métricas que GaitBase solo porque el optimizador descartó la rama de pose en todos los experimentos ($\alpha^*=1.0$). La silueta sola es suficiente y óptima para este dataset y ángulo de captura.